ERX
Zero to Hero

GIÁO TRÌNH BỔ TRỢ · PYTHON CƠ BẢN

Python cơ bản cho Demand Planner

Đọc, viết và kiểm chứng code dự báo trên dữ liệu Mekong Bloom Foods

Học phần	Demand Planning & Forecasting in Practice (ERX-DPF-101)
Tác giả	Thư Nguyễn (ERX)
Liên hệ	thu.minh090997@gmail.com
Đi kèm	9 notebook Jupyter (bản học viên + lời giải), dữ liệu MBF, thư viện dpkit
Yêu cầu	Không cần biết lập trình trước

Mục lục

Lời nói đầu

Chương 0 · Cài đặt và cách dùng notebook

Chương 1 · Python căn bản qua ví dụ Demand Planning

Chương 2 · pandas: đọc, lọc, gộp dữ liệu MBF

Chương 3 · Vẽ biểu đồ nhu cầu với matplotlib

Chương 4 · Làm sạch dữ liệu nhu cầu

Chương 5 · Dự báo mốc và đo sai số

Chương 6 · SES, Holt và dự báo cuốn chiếu

Chương 7 · Mùa vụ, Tết và hồi quy khuyến mãi

Chương 8 · Danh mục, FVA và tồn kho an toàn

Chương 9 · Dùng AI hỗ trợ viết code — an toàn và kiểm chứng

Phụ lục A · Bảng tra nhanh

Phụ lục B · Từ điển lỗi thường gặp

Phụ lục C · Lời giải bài tập

Lời nói đầu

Khóa học **không bắt buộc lập trình** — mọi bài thực hành đều làm được trên Excel và Forecasting Lab. Giáo trình này dành cho học viên muốn đi xa hơn: xử lý hàng trăm SKU một lúc, tự động hóa báo cáo hằng tháng, đọc hiểu và kiểm chứng code (kể cả code do AI viết).

Mỗi chương đi kèm một **notebook Jupyter** chạy được ngay trên dữ liệu Mekong Bloom Foods. Mọi đoạn code và kết quả in trong giáo trình được lấy từ chính notebook đã chạy thử, nên con số khớp với slide, file Excel và giáo trình chính.

Notebook	Nội dung	Giáo trình	Dùng cho
NB00_CaiDat_KiemTra	Cài đặt và kiểm tra môi trường	—	Trước Buổi 1
NB01_Python_CanBan	Python căn bản qua ví dụ Demand Planning	—	Trước Buổi 1
NB02_Pandas_DuLieu_MBF	pandas: đọc, lọc, gộp dữ liệu MBF	Chương 1	Buổi 1–2
NB03_BieuDo	Vẽ biểu đồ nhu cầu với matplotlib	Chương 1–2	Buổi 2
NB04_LamSach	Làm sạch dữ liệu nhu cầu	Chương 2	Buổi 2
NB05_Moc_SaiSo	Dự báo mốc và đo sai số	Chương 3	Buổi 3
NB06_SES_Holt	SES, Holt và dự báo cuốn chiếu	Chương 4	Buổi 4
NB07_MuaVu_HoiQuy	Mùa vụ, Tết và hồi quy khuyến mãi	Chương 5–6	Buổi 5–6
NB08_DanhMuc_FVA_TonKho	Danh mục, FVA và tồn kho an toàn	Chương 7–9	Buổi 7–9

Cách học hiệu quả

- Mở notebook bản học viên (thư mục notebooks), chạy từng ô theo thứ tự, đọc kết quả.
- Làm bài tập ở các ô “# Viết code của bạn ở đây”, rồi mới so với bản lời giải.
- Sửa một con số trong code (α, mã hàng, kênh) và chạy lại — cách nhanh nhất để hiểu code làm gì.
- Mỗi buổi chỉ cần 30–45 phút; không cần học hết trước khóa.

Chương 0 · Cài đặt và cách dùng notebook

0.1 Chọn một trong ba cách

Cách	Phù hợp khi	Các bước chính
Google Colab (không cần cài)	Máy công ty không cho cài phần mềm	Tải thư mục 10_Python_Co_Ban lên Google Drive → mở notebook bằng Colab → chạy ô kết nối Drive (0.3)
Anaconda + Jupyter	Máy cá nhân, muốn đủ công cụ ngay	Cài Anaconda → mở Anaconda Prompt → `cd` tới thư mục → `jupyter lab`
VS Code + Python	Đã quen VS Code	Cài Python 3.11+ và VS Code (extension Python, Jupyter) → `pip install -r requirements.txt` → mở file .ipynb

0.2 Cấu trúc thư mục

```
10_Python_Co_Ban/
├── notebooks/            ← bản học viên (làm bài ở đây)
├── notebooks_loi_giai/   ← bản lời giải (đã chạy sẵn, có kết quả)
├── data/                 ← 13 bảng CSV của Mekong Bloom Foods
├── dpkit/                ← thư viện tính toán của khóa (giống Forecasting Lab)
├── requirements.txt
└── ERX_DPF_Python_Co_Ban.pdf / .docx
```

Ô đầu tiên của mọi notebook tự tìm thư mục chứa **data/** và **dpkit/**, nên bạn có thể mở notebook ở bất kỳ đâu trong thư mục này.

Ô thiết lập

```
# Ô thiết lập — chạy ô này đầu tiên mỗi khi mở notebook
import sys, pathlib
HERE = pathlib.Path.cwd()
# tìm thư mục 10_Python_Co_Ban (nơi có data/ và dpkit/) — dù mở notebook ở thư mục nào
ROOT = next((p for p in [HERE, *HERE.parents] if (p / "data").exists() and (p /
"dpkit").exists()), HERE)
sys.path.insert(0, str(ROOT))                                  # để import được dpkit
DATA = ROOT / "data"
print("Thư mục dữ liệu:", DATA, "| tồn tại:", DATA.exists())
```

0.3 Dùng trên Google Colab

Chạy trước ô thiết lập khi dùng Colab

```
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/10_Python_Co_Ban/notebooks
```

Colab đã có sẵn pandas, numpy, matplotlib, scipy, statsmodels.

0.4 Cài thư viện trên máy

```
pip install -r requirements.txt
# hoặc: pip install pandas numpy matplotlib scipy statsmodels openpyxl jupyterlab
```

0.5 Thao tác cơ bản trong notebook

Thao tác	Phím tắt
Chạy ô hiện tại và sang ô sau	Shift + Enter
Chạy tất cả	Menu Run → Run All
Thêm ô bên dưới	B (khi không đang gõ)
Đổi ô sang văn bản / code	M / Y
Khởi động lại khi bị treo	Kernel → Restart

0.6 Chạy Forecasting Lab trên máy (tùy chọn)

```
cd 08_Forecasting_Lab
pip install -r requirements.txt
streamlit run app.py        # trình duyệt mở http://localhost:8501
```

0.7 Kiểm tra môi trường (NB00)

Kiểm tra phiên bản Python và các thư viện

Khóa học cần Python 3.10 trở lên và năm thư viện: pandas, numpy, matplotlib, scipy, statsmodels.

Code

```
import platform
print("Python", platform.python_version())
import pandas as pd, numpy as np, matplotlib, scipy, statsmodels
for m in (pd, np, matplotlib, scipy, statsmodels):
    print(f"{m.__name__:12s} {m.__version__}")
```

Đọc thử dữ liệu Mekong Bloom Foods

Code

```
sh = pd.read_csv(DATA / "sales_history.csv", encoding="utf-8-sig")
print(sh.shape)          # (số dòng, số cột)
sh.head()
```

Kiểm tra thư viện dpkit của khóa học

Code

```
from dpkit import metrics
print(metrics.wmape([100, 200], [110, 190]))   # kết quả đúng: 6.666…
```

Nếu cả bốn ô trên chạy không lỗi, bạn đã sẵn sàng. 🎉

Chương 1 · Python căn bản qua ví dụ Demand Planning

Notebook	notebooks/NB01_Python_CanBan.ipynb (lời giải: notebooks_loi_giai/NB01_Python_CanBan_LoiGiai.ipynb)
Giáo trình liên quan	—
Dùng cho	Trước Buổi 1

Mục tiêu: biến, kiểu dữ liệu, list, dict, if, for, hàm — đủ để đọc và sửa code trong khóa học.

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

1. Biến và kiểu dữ liệu

Biến là một cái tên gắn với một giá trị. Python tự nhận kiểu: số nguyên (int), số thực (float), chuỗi (str), đúng/sai (bool).

Code

```
sku = "N01"                 # chuỗi
gia_npp = 118.0             # số thực — nghìn đồng/thùng
so_thung = 46591            # số nguyên
dang_ban = True             # đúng/sai
doanh_thu = gia_npp * so_thung / 1000      # triệu đồng
print(sku, "doanh thu dự kiến:", doanh_thu, "triệu đồng")
print(type(sku), type(gia_npp), type(so_thung), type(dang_ban))
```

Kết quả

```
N01 doanh thu dự kiến: 5497.738 triệu đồng
<class 'str'> <class 'float'> <class 'int'> <class 'bool'>
```

f-string giúp in số đẹp. :,.0f là có dấu phân cách nghìn, không số lẻ. Hàm vn() dưới đây đổi sang kiểu Việt Nam (dấu chấm ngăn nghìn).

Code

```
def vn(x, d=0):
    """Định dạng số kiểu Việt Nam: 46.591 ; 3,98"""
    s = f"{x:,.{d}f}"
    return s.replace(",", "X").replace(".", ",").replace("X", ".")

print(f"Doanh thu: {doanh_thu:,.1f} tr.đ")
print("Doanh thu:", vn(doanh_thu, 1), "tr.đ")
```

Kết quả

```
Doanh thu: 5,497.7 tr.đ
Doanh thu: 5.497,7 tr.đ
```

2. List — danh sách có thứ tự

Chỉ số bắt đầu từ **0**. y[-1] là phần tử cuối; y[0:3] lấy 3 phần tử đầu.

Code

```
y = [5200, 5480, 5610, 5900, 6050, 6230]     # nhu cầu 6 tháng (thùng)
```

```
print("Tháng đầu:", y[0], "| tháng cuối:", y[-1], "| 3 tháng cuối:", y[-3:])
print("Tổng:", sum(y), "| Trung bình:", sum(y) / len(y))
y.append(6400)                                  # thêm tháng mới
print(len(y), "tháng")
```

Kết quả

```
Tháng đầu: 5200 | tháng cuối: 6230 | 3 tháng cuối: [5900, 6050, 6230]
Tổng: 34470 | Trung bình: 5745.0
7 tháng
```

3. Dict — từ điển khóa → giá trị

Rất hợp để tra cứu: mã hàng → giá, chuỗi → dự báo.

Code

```
gia = {"N01": 118.0, "S01": 372.0, "B01": 168.0}
print(gia["S01"])
gia["K01"] = 280.0                              # thêm khóa mới
for ma, g in gia.items():
    print(ma, g)
```

Kết quả

```
372.0
N01 118.0
S01 372.0
B01 168.0
K01 280.0
```

4. if — rẽ nhánh

Ví dụ: cảnh báo sớm trong tháng (Chương 9 giáo trình).

Code

```
du_bao_thang, ban_14_ngay = 7719, 2323
ty_le = ban_14_ngay / (du_bao_thang * 14 / 31)
if ty_le > 1.15:
    canh_bao = "Bán nhanh — nguy cơ đứt hàng"
elif ty_le < 0.85:
    canh_bao = "Bán chậm — nguy cơ tồn/cận date"
else:
    canh_bao = "Bình thường"
print(f"Tỷ lệ run-rate = {ty_le:.2f} → {canh_bao}")
```

Kết quả

```
Tỷ lệ run-rate = 0.67 → Bán chậm — nguy cơ tồn/cận date
```

5. for — lặp

Tính dự báo Naive và sai số tuyệt đối cho từng tháng.

Code

```
thuc_te = [100, 120, 110, 130]
```

```
sai_so = []
for t in range(1, len(thuc_te)):
    du_bao = thuc_te[t - 1]                  # Naive: dự báo = tháng trước
    sai_so.append(abs(thuc_te[t] - du_bao))
print("Sai số tuyệt đối:", sai_so, "| MAE =", sum(sai_so) / len(sai_so))
```

Kết quả

```
Sai số tuyệt đối: [20, 10, 20] | MAE = 16.666666666666668
```

Cách viết gọn hơn: **list comprehension**.

Code

```
sai_so = [abs(thuc_te[t] - thuc_te[t - 1]) for t in range(1, len(thuc_te))]
print(sai_so)
```

Kết quả

```
[20, 10, 20]
```

6. Hàm — đóng gói một phép tính để dùng lại

Code

```
def wmape(actual, forecast):
    """WMAPE (%) = Σ|thực tế − dự báo| / Σ thực tế × 100"""
    tong_sai = sum(abs(a - f) for a, f in zip(actual, forecast))
    return tong_sai / sum(actual) * 100

print(round(wmape([80, 120, 110], [100, 100, 100]), 1))    # 16.1
```

Kết quả

```
16.1
```

Bài tập

Bài tập 1.1 — Viết hàm bias_pct(actual, forecast) trả về Bias% = Σ(dự báo − thực tế) / Σ thực tế × 100. Thử với thực tế [80, 120, 110], dự báo [100, 100, 100] (đáp án −3,2).

Bài tập

Bài tập 1.2 — Cho gia = {'N01': 118, 'S01': 372} và sl = {'N01': 46591, 'S01': 10977}. Dùng vòng lặp in doanh thu (triệu đồng) từng mã và tổng.

7. Đọc thông báo lỗi

Lỗi là bình thường. Đọc **dòng cuối** của thông báo: NameError (gõ sai tên biến), KeyError (khóa/cột không có), IndexError (vượt độ dài list), TypeError (cộng chuỗi với số…). Ô dưới cố ý gây lỗi và bắt lỗi để bạn xem thông báo.

Code

```
try:
    print(gia["X99"])
```

```
except KeyError as e:
    print("KeyError — không có khóa:", e)
```

Kết quả

```
KeyError — không có khóa: 'X99'
```

Chương 2 · pandas: đọc, lọc, gộp dữ liệu MBF

Notebook	notebooks/NB02_Pandas_DuLieu_MBF.ipynb (lời giải: notebooks_loi_giai/NB02_Pandas_DuLieu_MBF_LoiGiai.ipynb)
Giáo trình liên quan	Chương 1
Dùng cho	Buổi 1–2

pandas là “Excel trong Python”: bảng dữ liệu gọi là **DataFrame**, một cột là **Series**.

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd
sh = pd.read_csv(DATA / "sales_history.csv", encoding="utf-8-sig")
sku = pd.read_csv(DATA / "sku_master.csv", encoding="utf-8-sig")
sh.info()
```

Kết quả

```
<class 'pandas.DataFrame'>
RangeIndex: 1536 entries, 0 to 1535
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   month          1536 non-null   str
 1   sku            1536 non-null   str
 2   channel        1536 non-null   str
 3   orders         1535 non-null   float64
 4   shipments      1535 non-null   float64
 5   returns        1536 non-null   float64
 6   net_shipments  1535 non-null   float64
 7   stockout_days  1536 non-null   int64
dtypes: float64(4), int64(1), str(3)
…
```

1. Xem nhanh dữ liệu

Code

```
print(sh.shape)
display(sh.head(3))
display(sh.describe().round(1))
```

Kết quả

```
(1536, 8)
```

Kết quả

```
     month  sku channel   orders  shipments  returns  net_shipments  \
0  2023-10  B01      GT   9030.0     9030.0      0.0         9030.0
1  2023-11  B01      GT  10818.0    10818.0      0.0        10818.0
2  2023-12  B01      GT   9463.0     9463.0      0.0         9463.0

   stockout_days
```

```
0             0
1             0
2             0
```

Kết quả

	orders	shipments	returns	net_shipments	stockout_days
count	1535.0	1535.0	1536.0	1535.0	1536.0
mean	5497.7	5493.4	1.5	5492.0	0.0
std	8232.0	8231.9	42.5	8233.0	0.6
min	0.0	0.0	0.0	-1398.0	0.0
25%	1499.5	1499.5	0.0	1499.5	0.0
50%	3342.0	3342.0	0.0	3342.0	0.0
75%	6331.5	6297.0	0.0	6297.0	0.0
max	136656.0	136656.0	1450.0	136656.0	15.0

2. Chọn cột và lọc dòng

Lọc giống AutoFilter của Excel: điều kiện trong ngoặc vuông, nối nhiều điều kiện bằng & (và) hoặc | (hoặc), mỗi điều kiện đặt trong ngoặc tròn.

Code

```
n01_gt = sh[(sh["sku"] == "N01") & (sh["channel"] == "GT")]
n01_gt[["month", "orders", "shipments", "stockout_days"]].tail()
```

Kết quả

	month	orders	shipments	stockout_days
751	2026-05	47562.0	47562.0	0
752	2026-06	51318.0	51318.0	0
753	2026-07	38545.0	38545.0	0
754	2026-08	41679.0	41679.0	0
755	2026-09	55051.0	55051.0	0

Code

```
# Những tháng có hết hàng
sh[sh["stockout_days"] > 0][["month", "sku", "channel", "shipments", "stockout_days"]]
```

Kết quả

	month	sku	channel	shipments	stockout_days
55	2025-05	B01	MT	5448.0	9
104	2026-06	B02	GT	10953.0	7
735	2025-01	N01	GT	34671.0	6
929	2026-03	N03	MT	3480.0	15
1165	2026-01	S01	MT	5654.0	5
1331	2024-11	S04	GT	3649.0	12

3. Tạo cột mới

Code

```
sh["fill_rate"] = sh["shipments"] / sh["orders"]
sh["year"] = sh["month"].str[:4]
sh[["month", "sku", "orders", "shipments", "fill_rate"]].head()
```

Kết quả

	month	sku	orders	shipments	fill_rate
0	2023-10	B01	9030.0	9030.0	1.0
1	2023-11	B01	10818.0	10818.0	1.0
2	2023-12	B01	9463.0	9463.0	1.0
3	2024-01	B01	7528.0	7528.0	1.0
4	2024-02	B01	10148.0	10148.0	1.0

4. Ghép bảng (giống VLOOKUP): merge

Code

```
x = sh.merge(sku[["sku", "name", "category", "price_k", "cost_k", "planning_item"]], on="sku",
how="left")
x["revenue_trieu"] = x["shipments"] * x["price_k"] / 1000
x[["month", "sku", "name", "category", "shipments", "revenue_trieu"]].head()
```

Kết quả

	month	sku	name	category	shipments	revenue_trieu
0	2023-10	B01	Trà Xanh Chanh 450ml	Đồ uống	9030.0	1517.040
1	2023-11	B01	Trà Xanh Chanh 450ml	Đồ uống	10818.0	1817.424
2	2023-12	B01	Trà Xanh Chanh 450ml	Đồ uống	9463.0	1589.784
3	2024-01	B01	Trà Xanh Chanh 450ml	Đồ uống	7528.0	1264.704
4	2024-02	B01	Trà Xanh Chanh 450ml	Đồ uống	10148.0	1704.864

5. Gộp nhóm: groupby (giống PivotTable)

Code

```
by_cat = x.groupby("category")["revenue_trieu"].sum().sort_values(ascending=False)
by_cat.round(0)
```

Kết quả

```
category
Gia vị            775527.0
Mì ăn liền        565405.0
Đồ uống           428738.0
Bánh kẹo & quà    181948.0
B2B / HoReCa       24220.0
Name: revenue_trieu, dtype: float64
```

Code

```
# Doanh thu theo năm × ngành
x.pivot_table(index="category", columns="year", values="revenue_trieu", aggfunc="sum").round(0)
```

Kết quả

year	2023	2024	2025	2026
category				
B2B / HoReCa	1342.0	9195.0	7338.0	6345.0
Bánh kẹo & quà	22936.0	59205.0	60936.0	38871.0
Gia vị	58068.0	252484.0	257004.0	207972.0
Mì ăn liền	41220.0	197091.0	184223.0	142871.0

```
Đồ uống         28220.0  138059.0  142600.0  119859.0
```

6. Chuỗi thời gian theo tháng

Đổi cột month sang kiểu Period để sắp xếp và tính toán theo tháng.

Code

```
s = (x[(x.planning_item == "N01") & (x.channel == "GT")]
     .groupby("month")["shipments"].sum())
s.index = pd.PeriodIndex(s.index, freq="M")
print(s.tail(6))
print("CV =", round(s.std() / s.mean(), 3))
```

Kết quả

```
month
2026-04    39838.0
2026-05    47562.0
2026-06    51318.0
2026-07    38545.0
2026-08    41679.0
2026-09    55051.0
Freq: M, Name: shipments, dtype: float64
CV = 0.177
```

7. CV ở các cấp (Chương 1)

Code

```
def cv(series):
    return series.std() / series.mean()

cap = {
    "SKU × kênh": x.groupby(["planning_item", "channel",
"month"]).shipments.sum().groupby(level=[0, 1]).apply(cv),
    "SKU": x.groupby(["planning_item", "month"]).shipments.sum().groupby(level=0).apply(cv),
    "Nhóm hàng": x.groupby(["category", "month"]).shipments.sum().groupby(level=0).apply(cv),
    "Toàn công ty": pd.Series([cv(x.groupby("month").shipments.sum())]),
}
pd.DataFrame({k: [v.median()] for k, v in cap.items()}, index=["CV trung vị"]).round(3)
```

Kết quả

```
             SKU × kênh    SKU  Nhóm hàng  Toàn công ty
CV trung vị       0.296  0.274       0.24         0.142
```

> **Bài tập**
> Bài tập 2.1 — Tính tổng xuất kho (thùng) theo kênh GT/MT cho năm 2025 (year == '2025').

> **Bài tập**
> Bài tập 2.2 — Liệt kê 5 mã kế hoạch có doanh thu 12 tháng gần nhất (10/2025–09/2026) lớn nhất.

8. Xuất ra Excel

Code

```
out = ROOT / "outputs"
out.mkdir(exist_ok=True)
by_cat.round(0).to_frame("Doanh thu (tr.đ)").to_excel(out / "doanh_thu_theo_nganh.xlsx")
print("Đã lưu:", out / "doanh_thu_theo_nganh.xlsx")
```

Kết quả

```
Đã lưu: .../10_Python_Co_Ban/outputs/doanh_thu_theo_nganh.xlsx
```

Chương 3 · Vẽ biểu đồ nhu cầu với matplotlib

Notebook	notebooks/NB03_BieuDo.ipynb (lời giải: notebooks_loi_giai/NB03_BieuDo_LoiGiai.ipynb)
Giáo trình liên quan	Chương 1–2
Dùng cho	Buổi 2

Biểu đồ là cách nhanh nhất để thấy xu hướng, mùa vụ và điểm bất thường. Nguyên tắc: **một trục**, ít màu, có tiêu đề nói thông điệp.

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

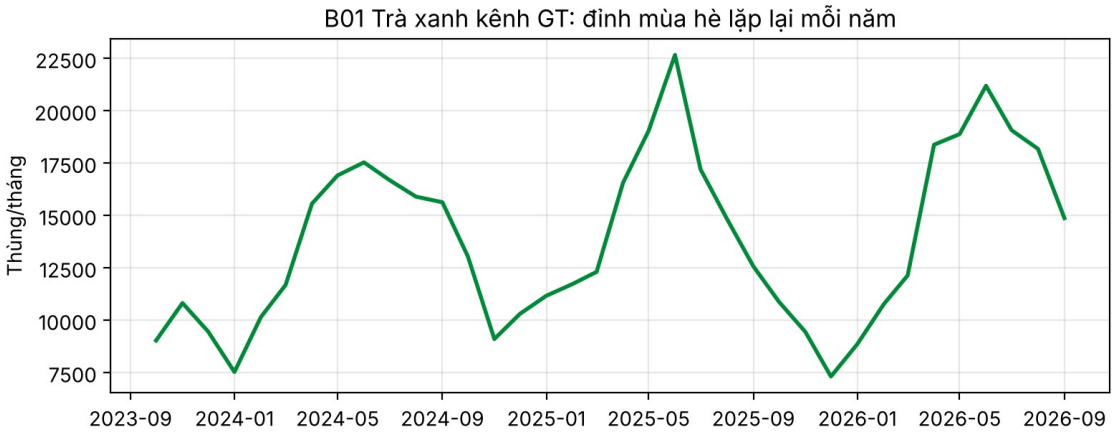
Code

```
import pandas as pd
import matplotlib.pyplot as plt
ERX = ["#008A3A", "#2563C9", "#D9720B", "#8E44AD", "#C2185B"]
dc = pd.read_csv(DATA / "demand_clean.csv", encoding="utf-8-sig")
def chuoi(item, channel, col="demand"):
    s = dc[(dc.item == item) & (dc.channel == channel)].set_index("month")[col]
    s.index = pd.PeriodIndex(s.index, freq="M")
    return s
b01 = chuoi("B01", "GT"); s01 = chuoi("S01", "GT")
```

1. Biểu đồ đường một chuỗi

Code

```
fig, ax = plt.subplots(figsize=(9, 3.2))
ax.plot(b01.index.to_timestamp(), b01.values, color=ERX[0], lw=2)
ax.set_title("B01 Trà xanh kênh GT: đỉnh mùa hè lặp lại mỗi năm")
ax.set_ylabel("Thùng/tháng")
ax.grid(alpha=0.3)
plt.show()
```

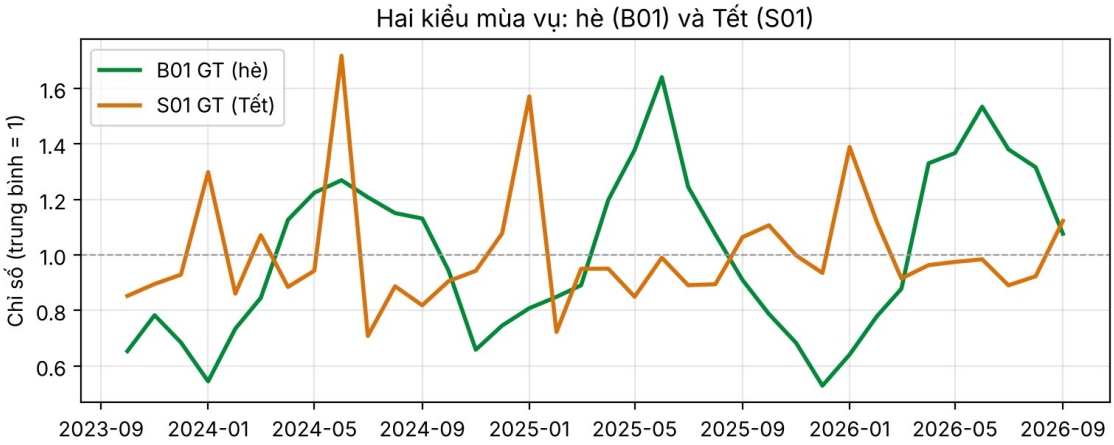


2. Nhiều chuỗi trên cùng trục — chuẩn hóa để so sánh hình dạng

Hai chuỗi khác quy mô: chia cho trung bình của chính nó (chỉ số = 1) rồi vẽ chung một trục — **không dùng hai trục y**.

Code

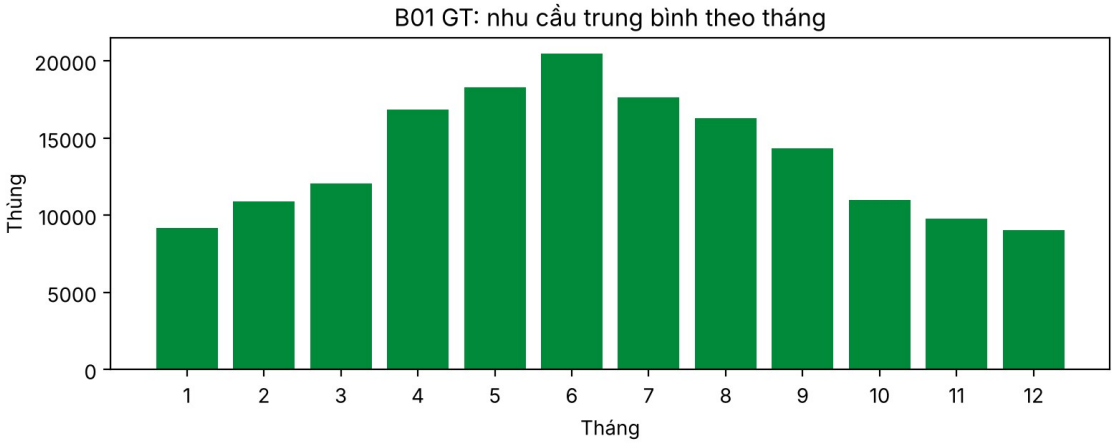
```
fig, ax = plt.subplots(figsize=(9, 3.2))
for s, ten, mau in [(b01, "B01 GT (hè)", ERX[0]), (s01, "S01 GT (Tết)", ERX[2])]:
    ax.plot(s.index.to_timestamp(), s / s.mean(), label=ten, color=mau, lw=2)
ax.axhline(1, color="#999", lw=0.8, ls="--")
ax.set_title("Hai kiểu mùa vụ: hè (B01) và Tết (S01)")
ax.set_ylabel("Chỉ số (trung bình = 1)")
ax.legend(); ax.grid(alpha=0.3); plt.show()
```



3. Biểu đồ cột: trung bình theo tháng trong năm

Code

```
thang = b01.groupby(b01.index.month).mean()
fig, ax = plt.subplots(figsize=(9, 3))
ax.bar(thang.index, thang.values, color=ERX[0])
ax.set_xticks(range(1, 13)); ax.set_xlabel("Tháng"); ax.set_ylabel("Thùng")
ax.set_title("B01 GT: nhu cầu trung bình theo tháng")
plt.show()
```



Bài tập

Bài tập 3.1 — Vẽ xuất kho (recorded) và nhu cầu đã làm sạch (demand) của N01 kênh GT trên cùng một biểu đồ. Bạn thấy gì ở các tháng 3, 6, 9, 12?

4. Lưu biểu đồ để đưa vào báo cáo

Code

```
out = ROOT / "outputs"; out.mkdir(exist_ok=True)
fig.savefig(out / "bieu_do.png", dpi=150, bbox_inches="tight")
print("Đã lưu", out / "bieu_do.png")
```

Kết quả

```
Đã lưu .../10_Python_Co_Ban/outputs/bieu_do.png
```

Chương 4 · Làm sạch dữ liệu nhu cầu

Notebook	notebooks/NB04_LamSach.ipynb (lời giải: notebooks_loi_giai/NB04_LamSach_LoiGiai.ipynb)
Giáo trình liên quan	Chương 2
Dùng cho	Buổi 2

Phát hiện bất thường, phục hồi nhu cầu khi hết hàng, nối mã, và **ghi nhật ký** — không sửa đè dữ liệu gốc.

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd, numpy as np
sh = pd.read_csv(DATA / "sales_history.csv", encoding="utf-8-sig")
sku = pd.read_csv(DATA / "sku_master.csv", encoding="utf-8-sig")
nhat_ky = []          # mỗi điều chỉnh là một dict → cuối cùng thành DataFrame
def ghi(series, month, goc, moi, quy_tac, ly_do, bang_chung):
    nhat_ky.append(dict(series=series, month=month, original=goc, adjusted=moi, rule=quy_tac,
reason=ly_do, evidence=bang_chung))
```

1. Phát hiện bất thường bằng IQR — N03 kênh GT

Code

```
n03 = sh[(sh.sku == "N03") & (sh.channel == "GT")].set_index("month")["shipments"]
q1, q3 = n03.quantile(0.25), n03.quantile(0.75)
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Ngưỡng: {lo:,.0f} – {hi:,.0f}")
n03[(n03 < lo) | (n03 > hi)]
```

Kết quả

```
Ngưỡng: 1,445 – 11,106
```

Kết quả

```
month
2024-08    136656.0
Name: shipments, dtype: float64
```

Tháng 08/2024 = 136.656 — gấp ~24 lần bình thường. Quy cách N03 là 24 ly/thùng → số được ghi theo ly.

Code

```
upc = int(sku.set_index("sku").loc["N03", "units_per_carton"])
goc = n03["2024-08"]; moi = round(goc / upc)
ghi("N03|GT", "2024-08", goc, moi, "Sửa đơn vị", f"Ghi theo ly, chia {upc}", "units_per_carton")
print(goc, "→", moi)
```

Kết quả

```
136656.0 → 5694
```

2. Robust z-score — bền hơn với điểm lỗi

Code

```
med = n03.median(); mad = (n03 - med).abs().median()
z = 0.6745 * (n03 - med) / mad
z[z.abs() > 3.5].round(1)
```

Kết quả

```
month
2024-08    78.7
Name: shipments, dtype: float64
```

3. Phục hồi nhu cầu khi hết hàng

- **MT**: chuỗi siêu thị vẫn ghi PO → dùng orders.
- **GT** (không có sell-out): quy đổi theo số ngày bán được.

Code

```
cal = pd.read_csv(DATA / "calendar.csv", encoding="utf-8-sig").set_index("month")
["days_in_month"]
het = sh[sh.stockout_days > 0].copy()
for r in het.itertuples():
    if r.channel == "MT":
        moi, quy_tac, ly_do = r.orders, "Phục hồi hết hàng", "Dùng PO của chuỗi (MT)"
    else:
        ngay = cal[r.month]
        moi, quy_tac, ly_do = round(r.shipments * ngay / (ngay - r.stockout_days)), "Phục hồi hết
hàng", f"Quy đổi theo {r.stockout_days} ngày hết hàng"
    ghi(f"{r.sku}|{r.channel}", r.month, r.shipments, moi, quy_tac, ly_do, "stockout_days")
pd.DataFrame(nhat_ky).tail(6)
```

Kết quả

```
   series    month  original  adjusted               rule  \
1  B01|MT  2025-05    5448.0    7677.0  Phục hồi hết hàng
2  B02|GT  2026-06   10953.0   14287.0  Phục hồi hết hàng
3  N01|GT  2025-01   34671.0   42992.0  Phục hồi hết hàng
4  N03|MT  2026-03    3480.0    6743.0  Phục hồi hết hàng
5  S01|MT  2026-01    5654.0    6741.0  Phục hồi hết hàng
6  S04|GT  2024-11    3649.0    6082.0  Phục hồi hết hàng

                         reason       evidence
1         Dùng PO của chuỗi (MT)  stockout_days
2   Quy đổi theo 7 ngày hết hàng  stockout_days
3   Quy đổi theo 6 ngày hết hàng  stockout_days
4         Dùng PO của chuỗi (MT)  stockout_days
5         Dùng PO của chuỗi (MT)  stockout_days
…
```

Lưu ý: N01, N02, S01 kênh GT có dữ liệu sell-out (DMS) — trong bản làm sạch của lớp, các chuỗi này dùng sell-out thay vì quy đổi theo ngày.

4. Nối mã cũ – mới (S06 → S06N)

Code

```
sh["item"] = sh["sku"].map(sku.set_index("sku")["planning_item"])
s06 = sh[(sh.item == "S06") & (sh.channel == "GT")].groupby("month")["shipments"].sum()
print(s06["2025-01":"2025-03"])
```

Kết quả

```
month
2025-01    3651.0
2025-02    2624.0
2025-03    3162.0
Name: shipments, dtype: float64
```

Bài tập

Bài tập 4.1 — S02 kênh MT bị thiếu dữ liệu tháng 2025-07 (ô trống). Nội suy tuyến tính giữa tháng trước và tháng sau, rồi ghi vào nhật ký.

5. Nhật ký điều chỉnh — xuất ra Excel

Code

```
log = pd.DataFrame(nhat_ky)
out = ROOT / "outputs"; out.mkdir(exist_ok=True)
log.to_excel(out / "nhat_ky_dieu_chinh.xlsx", index=False)
print(len(log), "dòng nhật ký"); log.groupby("rule").size()
```

Kết quả

```
8 dòng nhật ký
```

Kết quả

```
rule
Nội suy              1
Phục hồi hết hàng    6
Sửa đơn vị           1
dtype: int64
```

Chương 5 · Dự báo mốc và đo sai số

Notebook	notebooks/NB05_Moc_SaiSo.ipynb (lời giải: notebooks_loi_giai/NB05_Moc_SaiSo_LoiGiai.ipynb)
Giáo trình liên quan	Chương 3
Dùng cho	Buổi 3

Tự viết Naive, Seasonal Naive, MA(3) và 6 thước đo — rồi đối chiếu với thư viện dpkit.

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd, numpy as np
dc = pd.read_csv(DATA / "demand_clean.csv", encoding="utf-8-sig")
def chuoi(item, channel):
    s = dc[(dc.item == item) & (dc.channel == channel)].set_index("month")["demand"]
    s.index = pd.PeriodIndex(s.index, freq="M"); return s
y = chuoi("S01", "GT").values
train, test = y[:30], y[30:]          # 30 tháng train, 6 tháng test (04–09/2026)
print(len(train), len(test))
```

Kết quả

```
30 6
```

1. Ba dự báo mốc

Code

```
h = len(test)
naive = np.repeat(train[-1], h)
snaive = train[-12:][:h]                    # cùng tháng năm trước
ma3 = np.repeat(train[-3:].mean(), h)
pd.DataFrame({"Thực tế": test, "Naive": naive, "Seasonal Naive": snaive, "MA(3)": ma3.round()})
```

Kết quả

	Thực tế	Naive	Seasonal Naive	MA(3)
0	9555.0	9078.0	9428.0	11316.0
1	9669.0	9078.0	8426.0	11316.0
2	9764.0	9078.0	9820.0	11316.0
3	8834.0	9078.0	8838.0	11316.0
4	9157.0	9078.0	8873.0	11316.0
5	11134.0	9078.0	10556.0	11316.0

2. Sáu thước đo — viết từ công thức

Code

```
def do_sai_so(a, f, insample):
    e = a - f
    mae_naive_in = np.mean(np.abs(np.diff(insample)))
    return {"ME": e.mean(), "Bias%": (f.sum() - a.sum()) / a.sum() * 100, "MAE":
np.abs(e).mean(),
```

```
            "RMSE": np.sqrt((e ** 2).mean()), "MAPE%": np.mean(np.abs(e / a)) * 100,
            "WMAPE%": np.abs(e).sum() / a.sum() * 100, "MASE": np.abs(e).mean() / mae_naive_in}

bang = pd.DataFrame({ten: do_sai_so(test, f, train) for ten, f in
                     [("Naive", naive), ("Seasonal Naive", snaive), ("MA(3)", ma3)]}).T
bang.round(2)
```

Kết quả

	ME	Bias%	MAE	RMSE	MAPE%	WMAPE%	MASE
Naive	607.50	-6.27	688.83	943.43	6.70	7.11	0.29
Seasonal Naive	362.00	-3.74	382.00	574.32	3.85	3.94	0.16
MA(3)	-1630.83	16.84	1630.83	1783.43	17.45	16.84	0.70

Seasonal Naive có WMAPE ≈ **3,94%** — đúng như trong slide và Excel Buổi 3.

3. Đối chiếu với dpkit

Code

```
from dpkit import baseline, metrics
f = baseline.seasonal_naive(train, h=6)
pd.Series(metrics.summary(test, f, insample=train)).round(2)
```

Kết quả

```
ME        362.00
Bias%      -3.74
MAE       382.00
RMSE      574.32
MAPE%       3.85
WMAPE%      3.94
MASE        0.16
dtype: float64
```

4. Vì sao MAPE vô nghĩa với chuỗi gián đoạn — H02 kênh GT

Code

```
yh = chuoi("H02", "GT").values
trh, teh = yh[:30], yh[30:]
print("Thực tế 6 tháng test:", teh)
with np.errstate(divide="ignore", invalid="ignore"):
    print(pd.Series(do_sai_so(teh, trh[-12:][:6], trh)).round(2))
```

Kết quả

```
Thực tế 6 tháng test: [  0.  21. 199.   0.   0. 184.]
ME        -45.00
Bias%      66.83
MAE        65.00
RMSE      136.48
MAPE%        NaN
WMAPE%     96.53
MASE        0.37
dtype: float64
```

Bài tập

Bài tập 5.1 — Lặp qua 4 chuỗi N01|GT, N02|GT, B01|GT, S05|GT, tính WMAPE của 3 mốc và in mốc tốt nhất của từng chuỗi.

Chương 6 · SES, Holt và dự báo cuốn chiếu

Notebook	notebooks/NB06_SES_Holt.ipynb (lời giải: notebooks_loi_giai/NB06_SES_Holt_LoiGiai.ipynb)
Giáo trình liên quan	Chương 4
Dùng cho	Buổi 4

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd, numpy as np
dc = pd.read_csv(DATA / "demand_clean.csv", encoding="utf-8-sig")
def chuoi(item, channel):
    s = dc[(dc.item == item) & (dc.channel == channel)].set_index("month")["demand"]
    s.index = pd.PeriodIndex(s.index, freq="M"); return s
y = chuoi("N03", "GT").values
train, test = y[:30], y[30:]
```

1. SES viết bằng vòng lặp

L(t) = α·y(t) + (1−α)·L(t−1), khởi tạo L(1) = y(1). Dự báo 1 bước trong mẫu của tháng t là L(t−1).

Code

```
def ses_tay(y, alpha):
    L = y[0]; fitted = [np.nan]
    for t in range(1, len(y)):
        fitted.append(L)                    # dự báo cho tháng t = mức nền trước đó
        L = alpha * y[t] + (1 - alpha) * L
    return np.array(fitted), L              # L cuối = dự báo cho mọi tháng sau

fitted, L = ses_tay(train, 0.3)
sse = np.nansum((train - fitted) ** 2)
print(f"α = 0,3 → SSE = {sse:,.0f}; dự báo = {L:,.0f}")
```

Kết quả

```
α = 0,3 → SSE = 15,506,412; dự báo = 7,233
```

2. Chọn α tối ưu trên train (giống Solver)

Code

```
from scipy.optimize import minimize_scalar
res = minimize_scalar(lambda a: np.nansum((train - ses_tay(train, a)[0]) ** 2), bounds=(0.01,
0.99), method="bounded")
alpha = res.x
f_ses = np.repeat(ses_tay(train, alpha)[1], len(test))
wm = np.abs(test - f_ses).sum() / test.sum() * 100
print(f"α tối ưu = {alpha:.2f}; WMAPE test = {wm:.2f}%")
```

Kết quả

```
α tối ưu = 0.41; WMAPE test = 10.05%
```

3. Holt và Holt tắt dần với dpkit

Code

```
from dpkit import smoothing, metrics
for ten, r in [("SES", smoothing.ses(train, 6)), ("Holt", smoothing.holt(train, 6)), ("Holt tắt
dần", smoothing.holt(train, 6, damped=True))]:
    print(f"{ten:13s} WMAPE = {metrics.wmape(test, r.forecast):5.2f}%   tham số: { {k:
round(float(v), 2) for k, v in r.params.items() if isinstance(v, float)} }")
```

Kết quả

```
SES           WMAPE = 10.05%   tham số: {'alpha': 0.41}
Holt          WMAPE =  9.77%   tham số: {'alpha': 0.22, 'beta': 0.5, 'phi': 1.0}
Holt tắt dần  WMAPE =  8.43%   tham số: {'alpha': 0.19, 'beta': 0.5, 'phi': 0.84}
```

Holt tắt dần ≈ **8,43%**, tốt nhất cho N03 GT — chuỗi có xu hướng tăng.

4. Dự báo cuốn chiếu và sai số theo lag

Code

```
rows = []
for origin in range(24, len(y) - 3 + 1):          # đứng ở tháng origin, chỉ dùng dữ liệu đến đó
    fc = smoothing.holt(y[:origin], 3, damped=True).forecast
    for lag in (1, 2, 3):
        rows.append({"lag": lag, "actual": y[origin + lag - 1], "forecast": fc[lag - 1]})
ro = pd.DataFrame(rows)
ro.groupby("lag").apply(lambda d: metrics.wmape(d.actual, d.forecast)).round(2).rename("WMAPE %")
```

Kết quả

```
lag
1    12.16
2    12.58
3    12.08
Name: WMAPE %, dtype: float64
```

Bài tập

Bài tập 6.1 — Làm lại bước 2 cho N01 kênh GT. So WMAPE của SES với Naive. Mô hình nào thắng? Vì sao?

Chương 7 · Mùa vụ, Tết và hồi quy khuyến mãi

Notebook	notebooks/NB07_MuaVu_HoiQuy.ipynb (lời giải: notebooks_loi_giai/NB07_MuaVu_HoiQuy_LoiGiai.ipynb)
Giáo trình liên quan	Chương 5–6
Dùng cho	Buổi 5–6

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd, numpy as np
from dpkit import smoothing, metrics
dc = pd.read_csv(DATA / "demand_clean.csv", encoding="utf-8-sig")
cal = pd.read_csv(DATA / "calendar.csv", encoding="utf-8-sig").set_index("month")
def chuoi(item, channel, col="demand"):
    s = dc[(dc.item == item) & (dc.channel == channel)].set_index("month")[col]
    s.index = pd.PeriodIndex(s.index, freq="M"); return s
```

1. Chỉ số mùa vụ bằng trung bình trượt trung tâm 2×12

Code

```
b01 = chuoi("B01", "GT", "demand_ex_event")
cma = b01.rolling(12, center=True).mean().rolling(2).mean().shift(-1)    # CMA 2×12 căn giữa
ratio = (b01 / cma).dropna()
idx = ratio.groupby(ratio.index.month).mean()
idx = idx / idx.mean()                                                    # chuẩn hóa TB = 1
idx.round(2).rename("Chỉ số mùa vụ B01 GT")
```

Kết quả

```
month
1     0.72
2     0.80
3     0.87
4     1.18
5     1.32
6     1.49
7     1.26
8     1.14
9     1.04
10    0.88
11    0.68
12    0.63
Name: Chỉ số mùa vụ B01 GT, dtype: float64
```

2. Holt-Winters cộng và nhân

Code

```
y = chuoi("B01", "GT").values; train, test = y[:30], y[30:]
for mua in ("add", "mul"):
    r = smoothing.holt_winters(train, 6, 12, mua)
    print(f"HW {mua}: WMAPE = {metrics.wmape(test, r.forecast):.2f}%")
```

```
print(f"Seasonal Naive: {metrics.wmape(test, train[-12:][:6]):.2f}%")
```

Kết quả

```
HW add: WMAPE = 9.66%
HW mul: WMAPE = 3.83%
Seasonal Naive: 9.91%
```

3. Tết dịch tháng — phân bổ lại T12–T2 cho S01 GT

Code

```
print(cal.loc[["2026-12", "2027-01", "2027-02"], ["tet_pre_share", "tet_post_share"]])
muc_thuong, R, dip = 9887, 0.499, 0.15          # mức thường 2027 và phần Tết (giáo trình mục
5.4)
for m in ["2026-12", "2027-01", "2027-02"]:
    pre, post = cal.loc[m, "tet_pre_share"], cal.loc[m, "tet_post_share"]
    print(m, round(muc_thuong * (1 + R * pre) * (1 - dip * post)))
```

Kết quả

```
         tet_pre_share  tet_post_share
month
2026-12         0.0000             0.0
2027-01         0.8571             0.0
2027-02         0.1429             1.0
2026-12 9887
2027-01 14116
2027-02 9003
```

4. Hồi quy khuyến mãi K01 kênh MT với statsmodels

Code

```
import statsmodels.api as sm
k01 = chuoi("K01", "MT")
promo = pd.read_csv(DATA / "promo_calendar.csv", encoding="utf-8-sig")
p = promo[(promo.sku == "K01") & (promo.channel == "MT")].set_index("month")["depth"]
X = pd.DataFrame(index=k01.index.astype(str))
X["t"] = np.arange(1, len(X) + 1)
X["khai_giang"] = k01.index.month.isin([8, 9]).astype(int)
X["tet_pre"] = cal.loc[X.index, "tet_pre_share"].values
X["promo_depth"] = [p.get(m, 0.0) for m in X.index]
X["post_promo"] = (X["promo_depth"].shift(1, fill_value=0) > 0).astype(int)
model = sm.OLS(k01.values, sm.add_constant(X)).fit()
print(model.params.round(1)); print("R² =", round(model.rsquared, 3))
```

Kết quả

```
const          3245.1
t                37.3
khai_giang     1652.6
tet_pre        1144.0
promo_depth   10880.1
post_promo     -547.5
dtype: float64
R² = 0.824
```

Mỗi 10 điểm % giảm giá tăng ≈ **1.088** thùng; R^2 ≈ **0,82** — khớp slide Buổi 6.

Bài tập

Bài tập 7.1 — Baseline tháng 11/2026 của K01 MT ≈ 4.663 thùng. Với hệ số promo_depth vừa ước lượng, tính dự báo khi giảm 25% và uplift %.

Chương 8 · Danh mục, FVA và tồn kho an toàn

Notebook	notebooks/NB08_DanhMuc_FVA_TonKho.ipynb (lời giải: notebooks_loi_giai/NB08_DanhMuc_FVA_TonKho_LoiGiai.ipynb)
Giáo trình liên quan	Chương 7–9
Dùng cho	Buổi 7–9

Ô thiết lập (giống nhau ở mọi notebook) — xem Chương 0.

Code

```
import pandas as pd, numpy as np
from dpkit import intermittent, metrics, inventory
dc = pd.read_csv(DATA / "demand_clean.csv", encoding="utf-8-sig")
sku = pd.read_csv(DATA / "sku_master.csv", encoding="utf-8-
sig").drop_duplicates("planning_item").set_index("planning_item")
```

1. ABC theo giá trị 12 tháng

Code

```
last = dc[dc.month >= "2025-10"].groupby("item")["demand"].sum()
gt = (last * sku.loc[last.index, "price_k"] /
1000).sort_values(ascending=False).rename("gia_tri_trieu")
lk = gt.cumsum() / gt.sum()
abc = np.where(lk.shift(fill_value=0) < 0.8, "A", np.where(lk.shift(fill_value=0) < 0.95, "B",
"C"))
bang = pd.DataFrame({"gia_tri_trieu": gt.round(0), "luy_ke_%": (lk * 100).round(1), "ABC": abc})
bang.head(12)
```

Kết quả

item	gia_tri_trieu	luy_ke_%	ABC
S04	95212.0	13.9	A
N01	91170.0	27.1	A
S01	66145.0	36.8	A
B02	46697.0	43.6	A
B01	43254.0	49.8	A
N02	38830.0	55.5	A
N03	37304.0	60.9	A
S02	33356.0	65.8	A
B03	28780.0	70.0	A
S05	28358.0	74.1	A
K01	27814.0	78.1	A
K02	26263.0	82.0	A

2. Hình dạng nhu cầu (ADI, CV²) và Croston cho H02

Code

```
h02 = dc[(dc.item == "H02") & (dc.channel == "GT")]["demand"].values
adi, cv2 = intermittent.adi_cv2(h02)
print(f"ADI = {adi:.2f}; CV² = {cv2:.2f} → {intermittent.sbc_class(adi, cv2)}")
for v in ("croston", "sba", "tsb"):
```

```
    print(v, round(float(intermittent.croston(h02[:30], 6, 0.1, v)["forecast"][0]), 1),
"thùng/tháng")
```

Kết quả

```
ADI = 1.80; CV² = 0.40 → Intermittent (rời rạc)
croston 166.9 thùng/tháng
sba 158.5 thùng/tháng
tsb 171.8 thùng/tháng
```

3. FVA từ bảng forecast_history

Code

```
fh = pd.read_csv(DATA / "forecast_history.csv", encoding="utf-8-sig")
sh = pd.read_csv(DATA / "sales_history.csv", encoding="utf-8-sig")
sh["item"] = sh.sku.replace({"S06N": "S06"})
act = sh.groupby(["item", "channel", "month"]).shipments.sum().reset_index()
act.columns = ["sku", "channel", "origin_month", "naive"]            # Naive lag-1 = thực tế
tháng chốt
fh = fh.merge(act, on=["sku", "channel", "origin_month"], how="left").dropna(subset=["naive"])
fh = fh[fh.actual_shipments > 0]
a = fh.actual_shipments.values
pd.DataFrame(metrics.fva(a, {"Thống kê": fh.stat_forecast.values, "Sales": fh.sales_input.values,
                             "Đồng thuận": fh.consensus.values}, naive=fh.naive.values)).round(2)
```

Kết quả

	step	WMAPE%	Bias%	FVA_vs_prev_pp	vs
0	Naive	19.05	-0.14	NaN	NaN
1	Thống kê	17.49	-0.24	1.57	Naive
2	Sales	22.52	15.01	-5.04	Thống kê
3	Đồng thuận	18.24	7.84	4.28	Sales

4. Safety Stock và ROP

Code

```
ss = inventory.safety_stock(sigma_error=4512, lead_time_periods=2 / 4.33, service_level=0.95)
rop = inventory.reorder_point(46591, 2 / 4.33, ss)
print(f"N01 GT: SS ≈ {ss:,.0f} thùng; ROP ≈ {rop:,.0f} thùng")
pd.DataFrame([{"Mức phục vụ": sl, "SS": round(inventory.safety_stock(4512, 2 / 4.33, sl))} for sl
in (0.90, 0.95, 0.98, 0.99)])
```

Kết quả

```
N01 GT: SS ≈ 5,044 thùng; ROP ≈ 26,564 thùng
```

Kết quả

	Mức phục vụ	SS
0	0.90	3930
1	0.95	5044
2	0.98	6298
3	0.99	7134

Bài tập

Bài tập 8.1 — Tính FVA riêng cho các dòng có khuyến mãi (promo_flag == 1). Bước đồng thuận thêm bao nhiêu điểm % so với thống kê?

5. Xuất file bàn giao mẫu

Code

```
out = ROOT / "outputs"; out.mkdir(exist_ok=True)
pd.DataFrame({"series": ["N01|GT"], "month": ["2026-10"], "consensus_unconstrained": [46591],
"sigma_e": [4512], "safety_stock": [round(ss)]}).to_excel(out / "ban_giao_mau.xlsx", index=False)
print("Đã lưu", out / "ban_giao_mau.xlsx")
```

Kết quả

```
Đã lưu .../10_Python_Co_Ban/outputs/ban_giao_mau.xlsx
```

Chương 9 · Dùng AI hỗ trợ viết code — an toàn và kiểm chứng

Trợ lý AI (ChatGPT, Claude, Copilot…) giúp viết code rất nhanh — nhưng code chạy không lỗi **chưa chắc đúng**. Người chịu trách nhiệm con số vẫn là bạn.

Mẫu yêu cầu (prompt) hiệu quả

Ví dụ prompt

```
Bối cảnh: Tôi là Demand Planner. Tôi có file sales_history.csv với các cột
month (YYYY-MM), sku, channel (GT/MT), orders, shipments, stockout_days.
Việc cần làm: Viết code pandas tính nhu cầu đã phục hồi cho các tháng hết hàng:
- kênh MT: dùng orders; - kênh GT: shipments × số ngày / (số ngày − stockout_days).
Ràng buộc: không sửa cột gốc, tạo cột mới demand; ghi nhật ký mỗi điều chỉnh.
Kiểm tra: in 5 dòng đã điều chỉnh để tôi đối chiếu.
```

Danh sách kiểm chứng trước khi dùng kết quả

- ☐ Tính tay 2–3 dòng và so với kết quả code.
- ☐ So một con số tổng với file Excel Solution hoặc Forecasting Lab.
- ☐ Kiểm tra kích thước dữ liệu trước/sau (số dòng có bị mất do merge, dropna?).
- ☐ Kiểm tra giá trị thiếu (NaN) và giá trị 0 bất thường.
- ☐ Đọc lại từng dòng code mà bạn không hiểu — hỏi AI giải thích dòng đó.
- ☐ Không có dữ liệu test nào được dùng khi chọn tham số.

Dấu hiệu code có vấn đề

- Kết quả “quá đẹp” (WMAPE 1–2% cho chuỗi khó) — thường là rò rỉ dữ liệu tương lai.
- Số dòng thay đổi sau khi merge — khóa ghép không duy nhất.
- Cảnh báo SettingWithCopyWarning — có thể đang sửa một bản sao, không phải bảng gốc.
- Dùng hàm mà bạn không tìm thấy trong tài liệu chính thức — AI có thể bịa tên hàm.

Bảo mật dữ liệu

- Không dán dữ liệu thật chưa ẩn danh của công ty vào công cụ AI công cộng.
- Mô tả cấu trúc dữ liệu (tên cột, kiểu) thay vì dán dữ liệu; dùng dữ liệu MBF để thử.
- Tuân thủ chính sách AI của công ty bạn.

Phụ lục A · Bảng tra nhanh

Việc	pandas / numpy	Tương đương Excel
Đọc CSV	pd.read_csv(path, encoding='utf-8-sig')	Data → From Text/CSV
Xem 5 dòng đầu	df.head()	—
Lọc dòng	df[(df.sku == 'N01') & (df.channel == 'GT')]	AutoFilter
Tạo cột	df['x'] = df.a / df.b	Công thức cột mới
Ghép bảng	df.merge(other, on='sku', how='left')	VLOOKUP / XLOOKUP
Tổng theo nhóm	df.groupby('category')['x'].sum()	SUMIFS / PivotTable
Bảng xoay	df.pivot_table(index=…, columns=…, values=…, aggfunc='sum')	PivotTable
Sắp xếp	df.sort_values('x', ascending=False)	Sort
Lũy kế	s.cumsum()	=SUM(A2:A2)
Trung bình trượt	s.rolling(3).mean()	=AVERAGE(3 ô)
Lệch kỳ	s.shift(1)	Ô ngay trên
Độ lệch chuẩn, CV	s.std(), s.std() / s.mean()	STDEV.S, STDEV.S/AVERAGE
Phân vị	s.quantile(0.25)	QUARTILE.INC
Xuất Excel	df.to_excel('file.xlsx', index=False)	Save As

Phụ lục B · Từ điển lỗi thường gặp

Thông báo lỗi	Nguyên nhân thường gặp	Cách sửa
ModuleNotFoundError: No module named 'dpkit'	Chưa chạy ô thiết lập hoặc mở sai thư mục	Chạy ô thiết lập; kiểm tra in ra “tồn tại: True”
FileNotFoundError	Sai đường dẫn file dữ liệu	Dùng DATA / 'ten_file.csv'
KeyError: 'xxx'	Tên cột/khóa gõ sai hoặc không có	In df.columns để xem tên đúng
NameError	Dùng biến chưa tạo (chưa chạy ô trước)	Run All từ đầu
TypeError: unsupported operand	Cộng chuỗi với số	Đổi kiểu: float(x), str(x)
ValueError: shapes … not aligned	Hai mảng khác độ dài	Kiểm tra len() của từng mảng
UnicodeDecodeError	Sai bảng mã khi đọc CSV	encoding='utf-8-sig'

Phụ lục C · Lời giải bài tập

Lời giải đầy đủ kèm kết quả nằm trong thư mục notebooks_loi_giai. Hãy tự làm trước khi xem.

Chương 1 · Bài tập 1.1

```
def bias_pct(actual, forecast):
    return (sum(forecast) - sum(actual)) / sum(actual) * 100

print(round(bias_pct([80, 120, 110], [100, 100, 100]), 1))
```

Chương 1 · Bài tập 1.2

```
gia = {"N01": 118, "S01": 372}
sl = {"N01": 46591, "S01": 10977}
tong = 0
for ma in gia:
    dt = gia[ma] * sl[ma] / 1000
    tong += dt
    print(ma, vn(dt, 1), "tr.đ")
print("Tổng:", vn(tong, 1), "tr.đ")
```

Chương 2 · Bài tập 2.1

```
x[x.year == "2025"].groupby("channel")["shipments"].sum()
```

Chương 2 · Bài tập 2.2

```
last12 = x[x.month >= "2025-10"]
last12.groupby("planning_item")
["revenue_trieu"].sum().sort_values(ascending=False).head(5).round(0)
```

Chương 3 · Bài tập 3.1

```
rec = chuoi("N01", "GT", "recorded"); dem = chuoi("N01", "GT", "demand")
fig, ax = plt.subplots(figsize=(9, 3.2))
ax.plot(rec.index.to_timestamp(), rec, label="Xuất kho (ghi nhận)", color="#999")
ax.plot(dem.index.to_timestamp(), dem, label="Nhu cầu (sell-out)", color=ERX[0], lw=2)
ax.set_title("N01 GT: xuất kho bị đẩy lên cuối quý, nhu cầu thì không"); ax.legend();
ax.grid(alpha=0.3)
plt.show()
```

Chương 4 · Bài tập 4.1

```
s02 = sh[(sh.sku == "S02") & (sh.channel == "MT")].set_index("month")["shipments"]
print(s02["2025-06":"2025-08"])
moi = round((s02["2025-06"] + s02["2025-08"]) / 2)
ghi("S02|MT", "2025-07", np.nan, moi, "Nội suy", "Thiếu dữ liệu", "Ô trống trong ERP")
print("Nội suy:", moi)
```

Chương 5 · Bài tập 5.1

```
for key in ["N01|GT", "N02|GT", "B01|GT", "S05|GT"]:
    it, ch = key.split("|"); yy = chuoi(it, ch).values; tr, te = yy[:30], yy[30:]
    w = {"Naive": metrics.wmape(te, np.repeat(tr[-1], 6)),
         "Seasonal Naive": metrics.wmape(te, tr[-12:][:6]),
         "MA(3)": metrics.wmape(te, np.repeat(tr[-3:].mean(), 6))}
    best = min(w, key=w.get)
    print(f"{key}: tốt nhất {best} ({w[best]:.2f}%)")
```

Chương 6 · Bài tập 6.1

```
y1 = chuoi("N01", "GT").values; tr1, te1 = y1[:30], y1[30:]
a1 = minimize_scalar(lambda a: np.nansum((tr1 - ses_tay(tr1, a)[0]) ** 2), bounds=(0.01, 0.99),
method="bounded").x
print(f"α = {a1:.2f}; SES {metrics.wmape(te1, np.repeat(ses_tay(tr1, a1)[1], 6)):.2f}% vs Naive
{metrics.wmape(te1, np.repeat(tr1[-1], 6)):.2f}%")
# Naive thắng: chuỗi ổn định, thay đổi chậm — α tối ưu rất cao (gần Naive)
```

Chương 7 · Bài tập 7.1

```
b = 4663
f = b + model.params["promo_depth"] * 0.25
print(f"Dự báo ≈ {f:,.0f} thùng; uplift ≈ {(f / b - 1) * 100:.0f}%  (Marketing kỳ vọng 110%)")
```

Chương 8 · Bài tập 8.1

```
d = fh[fh.promo_flag == 1]
w_stat = metrics.wmape(d.actual_shipments, d.stat_forecast); w_cons =
metrics.wmape(d.actual_shipments, d.consensus)
print(f"Có KM: thống kê {w_stat:.2f}% → đồng thuận {w_cons:.2f}% (cải thiện {w_stat - w_cons:.2f}
điểm %)")
```